

Card Fraud Intelligence — Model Training

This notebook trains an XGBoost fraud detection model on the **IEEE-CIS Fraud Detection** dataset using SHAP for explainability.

Before you start

1. Upload the `card-fraud-intelligence/` project folder to your Google Drive
2. Enable GPU: **Runtime** → **Change runtime type** → **T4 GPU**
3. Run all cells top to bottom

Output: `models/fraud_detector.pkl` and `models/shap_explainer.pkl` saved to your Drive

Step 1 — Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Step 2 — Set Project Path

Update `PROJECT_PATH` to wherever you uploaded the project folder in your Drive.

```
import os
import sys

# UPDATE THIS to match your Drive folder location
PROJECT_PATH = '/content/drive/MyDrive/card-fraud-intelligence'

os.chdir(PROJECT_PATH)
sys.path.insert(0, PROJECT_PATH)

print(f'Working directory: {os.getcwd()}')
print('Files found:', os.listdir('.'))
```

Working directory: /content/drive/MyDrive/card-fraud-intelligence
Files found: ['.gitignore', '.env.example', 'requirements.txt', 'README.md', '.DS_Store', 'QUICKSTART.txt', 'data']

Step 3 — Install Dependencies

```
%%capture
!pip install xgboost shap scikit-learn imbalanced-learn pandas numpy matplotlib seaborn joblib
```

```
import xgboost as xgb
import shap
import torch

gpu_available = torch.cuda.is_available()
print(f'XGBoost version : {xgb.__version__}')
print(f'SHAP version    : {shap.__version__}')
print(f'GPU available   : {gpu_available}')
if gpu_available:
    print(f'GPU device      : {torch.cuda.get_device_name(0)}')
```

XGBoost version : 3.2.0
SHAP version : 0.52.0
GPU available : True
GPU device : Tesla T4

Step 4 — Verify Dataset

```
import pandas as pd

DATA_DIR = 'ieee-fraud-detection'

txn_path = f'{DATA_DIR}/train_transaction.csv'
id_path  = f'{DATA_DIR}/train_identity.csv'
```

```

assert os.path.exists(txn_path), f'Not found: {txn_path}'
print(f'train_transaction.csv found')

# Quick peek – load just 5 rows to check
sample = pd.read_csv(txn_path, nrows=5)
print(f'Columns          : {len(sample.columns)}')
print(f'Sample columns   : {list(sample.columns[:8])} ...')
print(f'Identity file    : {"found" if os.path.exists(id_path) else "not found (optional)"}')

train_transaction.csv found
Columns          : 394
Sample columns   : ['TransactionID', 'isFraud', 'TransactionDT', 'TransactionAmt', 'ProductCD', 'card1', 'card2',
Identity file    : found

```

▼ Step 5 — Load & Engineer Features

```

import numpy as np
import warnings
warnings.filterwarnings('ignore')

from training.train_ieee import load_and_engineer

df, feature_cols = load_and_engineer(DATA_DIR)

print(f'\nDataset shape      : {df.shape}')
print(f'Fraud rate           : {df["isFraud"].mean():.3%}')
print(f'Fraud count          : {df["isFraud"].sum():,}')
print(f'Legit count           : {(df["isFraud"] == 0).sum():,}')
print(f'Features used         : {len(feature_cols)}')

```

```

Dataset shape      : (590540, 439)
Fraud rate         : 3.499%
Fraud count        : 20,663
Legit count        : 569,877
Features used      : 245

```

▼ Step 6 — Exploratory Analysis

```

import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

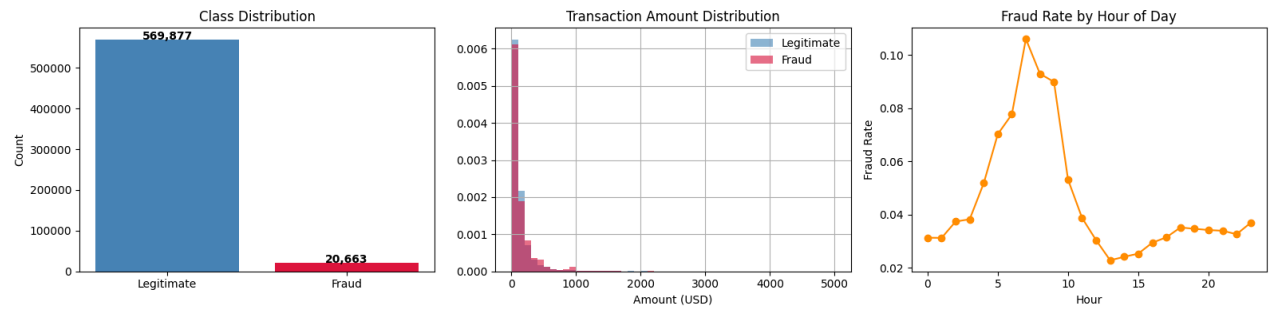
# Class distribution
counts = df['isFraud'].value_counts()
axes[0].bar(['Legitimate', 'Fraud'], counts.values, color=['steelblue', 'crimson'])
axes[0].set_title('Class Distribution')
axes[0].set_ylabel('Count')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 500, f'{v:,}', ha='center', fontweight='bold')

# Transaction amount distribution
df[df['isFraud'] == 0]['TransactionAmt'].clip(upper=5000).hist(
    bins=50, ax=axes[1], alpha=0.6, color='steelblue', label='Legitimate', density=True)
df[df['isFraud'] == 1]['TransactionAmt'].clip(upper=5000).hist(
    bins=50, ax=axes[1], alpha=0.6, color='crimson', label='Fraud', density=True)
axes[1].set_title('Transaction Amount Distribution')
axes[1].set_xlabel('Amount (USD)')
axes[1].legend()

# Hour of day
df.groupby('hour')['isFraud'].mean().plot(ax=axes[2], color='darkorange', marker='o')
axes[2].set_title('Fraud Rate by Hour of Day')
axes[2].set_xlabel('Hour')
axes[2].set_ylabel('Fraud Rate')

plt.tight_layout()
plt.savefig('models/eda_overview.png', dpi=150, bbox_inches='tight')
plt.show()
print('EDA saved to models/eda_overview.png')

```



EDA saved to models/eda_overview.png

Step 7 — Train XGBoost Model (5-Fold CV)

```
import joblib
from pathlib import Path
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, average_precision_score
from xgboost import XGBClassifier

X = df[feature_cols].values.astype(np.float32)
y = df['isFraud'].values

fraud_count = y.sum()
legit_count = len(y) - fraud_count
scale_pos_weight = legit_count / fraud_count
print(f'scale_pos_weight : {scale_pos_weight:.2f}')

# XGBoost 2.0+: GPU is enabled via device='cuda', not tree_method='gpu_hist'
device = 'cuda' if gpu_available else 'cpu'
print(f'device : {device}')

model = XGBClassifier(
    n_estimators=500,
    max_depth=7,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.5,
    scale_pos_weight=scale_pos_weight,
    eval_metric='aucpr',
    random_state=42,
    n_jobs=-1,
    tree_method='hist',
    device=device,
)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_aucs, cv_aps = [], []

print('\nStarting 5-fold cross validation ...')
print('-' * 50)

for fold, (tr_idx, val_idx) in enumerate(skf.split(X, y), 1):
    X_tr, X_val = X[tr_idx], X[val_idx]
    y_tr, y_val = y[tr_idx], y[val_idx]

    model.fit(
        X_tr, y_tr,
        eval_set=[(X_val, y_val)],
        verbose=100,
    )

    preds = model.predict_proba(X_val)[:, 1]
    auc = roc_auc_score(y_val, preds)
    ap = average_precision_score(y_val, preds)
    cv_aucs.append(auc)
    cv_aps.append(ap)
    print(f'Fold {fold} | AUC-R0C: {auc:.4f} | Avg Precision: {ap:.4f}')

print('-' * 50)
print(f'CV Mean AUC-R0C : {np.mean(cv_aucs):.4f} ± {np.std(cv_aucs):.4f}')
```

```
print(f'CV Mean Avg Precision : {np.mean(cv_aps):.4f} ± {np.std(cv_aps):.4f}')
```

```
scale_pos_weight : 27.58
device           : cuda
```

```
Starting 5-fold cross validation ...
```

```
-----
[0]      validation_0-aucpr:0.38659
[100]    validation_0-aucpr:0.59424
[200]    validation_0-aucpr:0.64791
[300]    validation_0-aucpr:0.67858
[400]    validation_0-aucpr:0.69923
[499]    validation_0-aucpr:0.71314
Fold 1 | AUC-ROC: 0.9502 | Avg Precision: 0.7132
[0]      validation_0-aucpr:0.37884
[100]    validation_0-aucpr:0.59714
[200]    validation_0-aucpr:0.65139
[300]    validation_0-aucpr:0.68103
[400]    validation_0-aucpr:0.70410
[499]    validation_0-aucpr:0.71955
Fold 2 | AUC-ROC: 0.9523 | Avg Precision: 0.7196
[0]      validation_0-aucpr:0.39724
[100]    validation_0-aucpr:0.59665
[200]    validation_0-aucpr:0.64592
[300]    validation_0-aucpr:0.67535
[400]    validation_0-aucpr:0.69553
[499]    validation_0-aucpr:0.71055
Fold 3 | AUC-ROC: 0.9501 | Avg Precision: 0.7106
[0]      validation_0-aucpr:0.37512
[100]    validation_0-aucpr:0.59849
[200]    validation_0-aucpr:0.64683
[300]    validation_0-aucpr:0.67585
[400]    validation_0-aucpr:0.69797
[499]    validation_0-aucpr:0.71320
Fold 4 | AUC-ROC: 0.9520 | Avg Precision: 0.7132
[0]      validation_0-aucpr:0.33730
[100]    validation_0-aucpr:0.58914
[200]    validation_0-aucpr:0.64209
[300]    validation_0-aucpr:0.66994
[400]    validation_0-aucpr:0.69152
[499]    validation_0-aucpr:0.70640
Fold 5 | AUC-ROC: 0.9513 | Avg Precision: 0.7064
-----
```

```
CV Mean AUC-ROC      : 0.9512 ± 0.0009
CV Mean Avg Precision : 0.7126 ± 0.0043
```

✓ Step 8 — Train Final Model on Full Dataset

```
print('Training final model on full dataset ...')
model.fit(X, y, verbose=100)

final_preds = model.predict_proba(X)[: , 1]
print(f'\nFinal AUC-ROC      : {roc_auc_score(y, final_preds):.4f}')
print(f'Final Avg Precision : {average_precision_score(y, final_preds):.4f}')
```

```
Training final model on full dataset ...
```

```
Final AUC-ROC      : 0.9777
Final Avg Precision : 0.7955
```

✓ Step 9 — Build SHAP Explainer

```
print('Building SHAP TreeExplainer (background = 5000 samples) ...')
background_idx = np.random.choice(len(X), size=5000, replace=False)
background = X[background_idx]

explainer = shap.TreeExplainer(model, background)
print('SHAP explainer ready.')
```

```
WARNING:shap:Background dataset has 5000 samples but max_samples=100. Subsampling to 100 samples for SHAP value computation.
Building SHAP TreeExplainer (background = 5000 samples) ...
SHAP explainer ready.
```

✓ Step 10 — Save Model Files to Drive

```
Path('models').mkdir(exist_ok=True)

MODEL_PATH = 'models/fraud_detector.pkl'
```

```

EXPLAINER_PATH = 'models/shap_explainer.pkl'
FEATURES_PATH = 'models/fraud_detector_feature_cols.pkl'

joblib.dump(model, MODEL_PATH)
joblib.dump(explainer, EXPLAINER_PATH)
joblib.dump(feature_cols, FEATURES_PATH)

print(f'Saved: {MODEL_PATH} ({Path(MODEL_PATH).stat().st_size / 1e6:.1f} MB)')
print(f'Saved: {EXPLAINER_PATH} ({Path(EXPLAINER_PATH).stat().st_size / 1e6:.1f} MB)')
print(f'Saved: {FEATURES_PATH}')
print('\nAll files saved to your Google Drive.')

```

```

Saved: models/fraud_detector.pkl (3.4 MB)
Saved: models/shap_explainer.pkl (12.8 MB)
Saved: models/fraud_detector_feature_cols.pkl

```

All files saved to your Google Drive.

▼ Step 11 — Evaluation Plots

```

from sklearn.metrics import roc_curve, precision_recall_curve, confusion_matrix, classification_report

THRESHOLD = 0.5
binary_preds = (final_preds >= THRESHOLD).astype(int)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

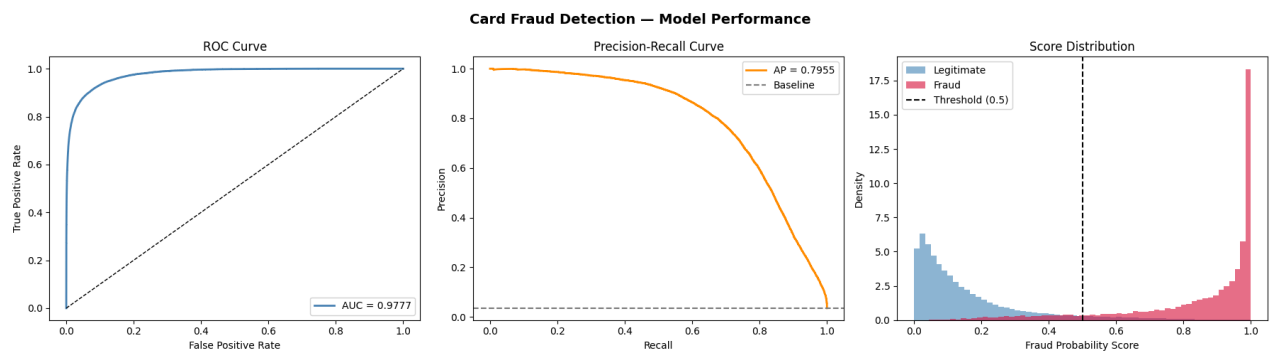
# ROC Curve
fpr, tpr, _ = roc_curve(y, final_preds)
auc_score = roc_auc_score(y, final_preds)
axes[0].plot(fpr, tpr, color='steelblue', lw=2, label=f'AUC = {auc_score:.4f}')
axes[0].plot([0,1],[0,1], 'k--', lw=1)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('ROC Curve')
axes[0].legend()

# Precision-Recall Curve
precision, recall, _ = precision_recall_curve(y, final_preds)
ap_score = average_precision_score(y, final_preds)
axes[1].plot(recall, precision, color='darkorange', lw=2, label=f'AP = {ap_score:.4f}')
axes[1].axhline(y.mean(), color='gray', linestyle='--', label='Baseline')
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curve')
axes[1].legend()

# Score Distribution
axes[2].hist(final_preds[y==0], bins=60, alpha=0.6, color='steelblue', label='Legitimate', density=True)
axes[2].hist(final_preds[y==1], bins=60, alpha=0.6, color='crimson', label='Fraud', density=True)
axes[2].axvline(THRESHOLD, color='black', linestyle='--', label=f'Threshold ({THRESHOLD})')
axes[2].set_xlabel('Fraud Probability Score')
axes[2].set_ylabel('Density')
axes[2].set_title('Score Distribution')
axes[2].legend()

plt.suptitle('Card Fraud Detection - Model Performance', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('models/evaluation_plots.png', dpi=150, bbox_inches='tight')
plt.show()
print('Plots saved to models/evaluation_plots.png')

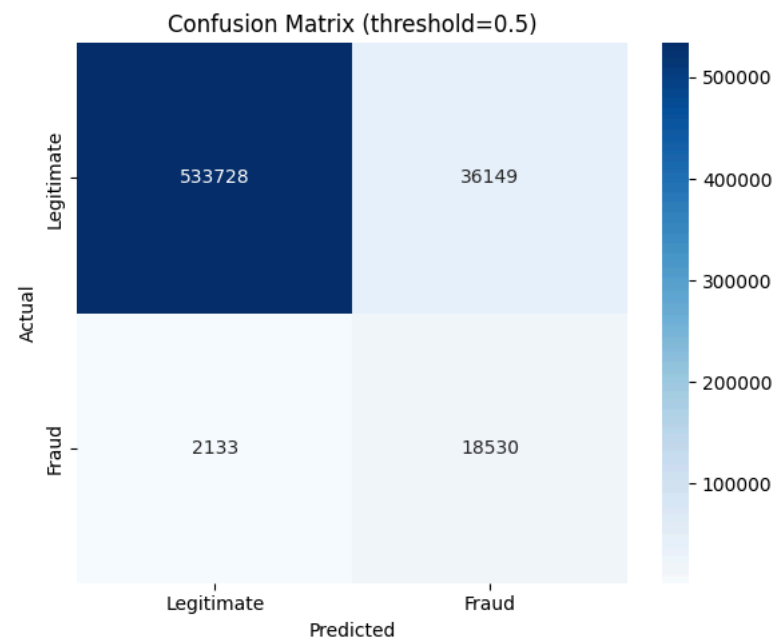
```



Plots saved to models/evaluation_plots.png

```
cm = confusion_matrix(y, binary_preds)
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['Legitimate', 'Fraud'],
    yticklabels=['Legitimate', 'Fraud']
)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title(f'Confusion Matrix (threshold={THRESHOLD})')
plt.tight_layout()
plt.show()

print('\nClassification Report:')
print(classification_report(y, binary_preds, target_names=['Legitimate', 'Fraud']))
```



Classification Report:

	precision	recall	f1-score	support
Legitimate	1.00	0.94	0.97	569877
Fraud	0.34	0.90	0.49	20663
accuracy			0.94	590540
macro avg	0.67	0.92	0.73	590540
weighted avg	0.97	0.94	0.95	590540

Step 12 — SHAP Feature Importance & Waterfall Plot

```
# SHAP values on a 2000-sample subset (full dataset is too slow to visualise)
print('Computing SHAP values on 2000 samples ...')
sample_idx = np.random.choice(len(X), size=2000, replace=False)
```

```

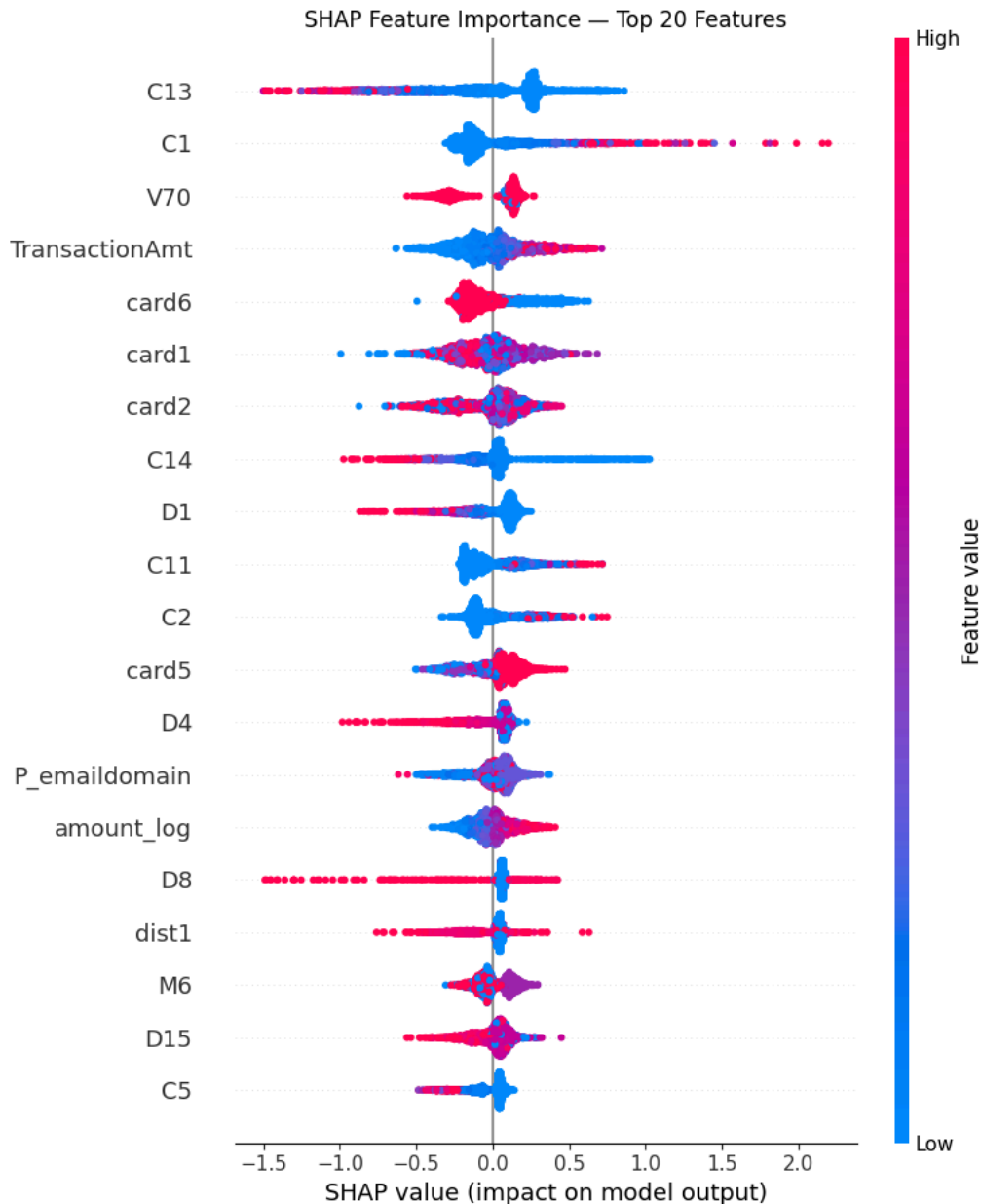
X_sample = X[sample_idx]
shap_values = explainer.shap_values(X_sample)

# Summary (beeswarm) plot
plt.figure()
shap.summary_plot(
    shap_values, X_sample,
    feature_names=feature_cols,
    max_display=20,
    show=False
)
plt.title('SHAP Feature Importance — Top 20 Features')
plt.tight_layout()
plt.savefig('models/shap_summary.png', dpi=150, bbox_inches='tight')
plt.show()
print('SHAP summary plot saved to models/shap_summary.png')

```

Computing SHAP values on 2000 samples ...

99%|=====| 1985/2000 [01:03<00:00]



SHAP summary plot saved to models/shap_summary.png

```

# Waterfall plot for the highest-scoring fraud in the sample
sample_probs = model.predict_proba(X_sample)[: , 1]
top_fraud_idx = np.argmax(sample_probs)

print(f'Showing SHAP waterfall for sample with fraud prob = {sample_probs[top_fraud_idx]:.4f}')

shap.plots.waterfall(
    shap.Explanation(
        values=shap_values[top_fraud_idx],
        base_values=explainer.expected_value,
        data=X_sample[top_fraud_idx],
    )
)

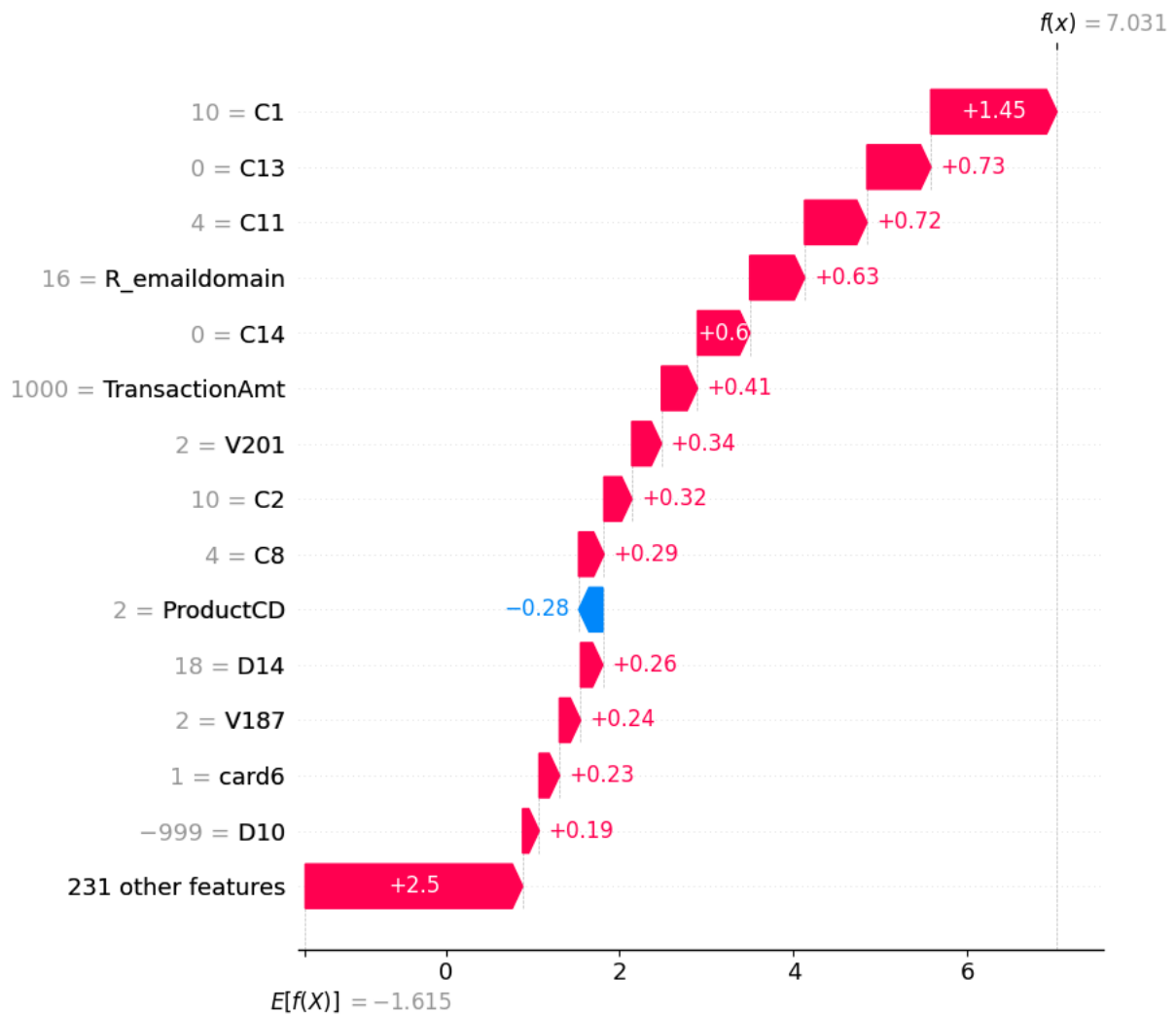
```

```

        feature_names=feature_cols,
    ),
    max_display=15,
    show=True,
)

```

Showing SHAP waterfall for sample with fraud prob = 0.9991



Step 13 — Test the Model (Simulate API Scoring)

```

# Verify the saved IEEE model works correctly.
#
# Important: this model was trained on 245 IEEE-CIS features produced by
# load_and_engineer(). It is NOT compatible with the FastAPI app's
# build_features() which produces 21 real-time behavioural features.
# The FastAPI app uses a separate model trained on synthetic data (training/train.py).

loaded_model = joblib.load(MODEL_PATH)

# Sample a mix of fraud and legit rows from the already-engineered df
rng = np.random.default_rng(42)
sample_idx = rng.choice(len(df), size=min(5000, len(df)), replace=False)
X_sample = df.iloc[sample_idx][feature_cols].values.astype(np.float32)
y_sample = df.iloc[sample_idx]['isFraud'].values

probs = loaded_model.predict_proba(X_sample)[: , 1]

# Show the highest-scoring transaction in the sample
top_i = np.argmax(probs)
row = df.iloc[sample_idx[top_i]]
fraud_prob = probs[top_i]

print('Highest-scoring transaction in sample:')
print(f' Amount : ${row["TransactionAmt"]:.2f}')
print(f' True Label : {"FRAUD" if row["isFraud"] == 1 else "Legitimate"}')
print(f' Fraud Score : {fraud_prob:.4f}')
print(f' Risk Level : {"CRITICAL" if fraud_prob >= 0.85 else "HIGH" if fraud_prob >= 0.65 else "MEDIUM" if fr')
print(f' Decision : {"FLAGGED" if fraud_prob >= 0.5 else "CLEAR"}')

```

```

print()

n_fraud = y_sample.sum()
recall = (probs[y_sample == 1] >= 0.5).mean() if n_fraud > 0 else 0.0
specificity = (probs[y_sample == 0] < 0.5).mean()

print(f'Sanity check on {len(X_sample):,}-row sample ({n_fraud} fraud, {len(X_sample)-n_fraud} legit):')
print(f'  Recall @ 0.5      : {recall:.1%}')
print(f'  Specificity        : {specificity:.1%}')
print(f'  Feature shape       : {X_sample.shape[1]} (matches training ✓)')

```

Highest-scoring transaction in sample:

```

Amount      : $480.00
True Label  : FRAUD
Fraud Score : 0.9996
Risk Level  : CRITICAL
Decision    : FLAGGED

```

Sanity check on 5,000-row sample (172 fraud, 4828 legit):

```

Recall @ 0.5 : 90.7%
Specificity   : 94.8%
Feature shape : 245 (matches training ✓)

```

Done!

Your model files are saved to Google Drive:

```

card-fraud-intelligence/
├─ models/
│   ├── fraud_detector.pkl          ← Load in FastAPI app
│   ├── shap_explainer.pkl          ← Powers explanations
│   ├── fraud_detector_feature_cols.pkl
│   ├── evaluation_plots.png        ← ROC / PR / Score distribution
│   ├── shap_summary.png            ← Feature importance
│   └─ eda_overview.png             ← Dataset overview

```

Next steps